

1. Estados en React

¿Qué es el Estado en React?

El **Estado** (*state*) es un objeto o valor interno gestionado directamente por un componente que almacena datos dinámicos que pueden cambiar a lo largo del ciclo de vida de la aplicación. Mientras que las *props* se reciben de forma externa e inmutable desde un componente padre, el estado es privado y controlado completamente por el propio componente.

Características principales

- **Desencadenador de renderizado:** Cada vez que el estado cambia mediante su función actualizadora, React vuelve a renderizar de forma automática el componente (y sus hijos) para reflejar los nuevos datos en la interfaz.
- **Inmutabilidad:** El estado no debe modificarse directamente (por ejemplo: estado = nuevoValor es incorrecto). Siempre debe actualizarse utilizando la función despachadora correspondiente (como setEstado).
- **Persistencia entre renderizados:** A diferencia de una variable local común que se reinicializa cada vez que la función del componente se ejecuta, el estado conserva su valor entre las distintas ejecuciones de renderizado.
- **Asincronía y optimización:** Las actualizaciones de estado pueden agruparse en lotes (*batching*) por React para mejorar el rendimiento global de la interfaz gráfica.

¿Para qué sirve?

Sirve para dar interactividad a la interfaz, permitiendo que la aplicación responda a acciones del usuario (escritura en campos de texto, clics en botones, apertura de menús desplegables) o a eventos externos (respuestas de APIs y temporizadores).

```
import React, { useState } from 'react';
function ContadorInteractivo() {
  const [contador, setContador] = useState(0);
  const incrementar = () => {
    setContador(contador + 1);
  };
  const reiniciar = () => {
    setContador(0);
  };
  return (
    <div className="card-contador">
      <h3>Contador de Visitas</h3>
      <p>Valor actual: <strong>{contador}</strong></p>
      <button onClick={incrementar}>Incrementar</button>
      <button onClick={reiniciar}>Reiniciar</button>
    </div>
  );
}
export default ContadorInteractivo;
```

Explicación del ejemplo:

1. Se inicializa el hook `useState(0)`, asignando el valor inicial 0 a la variable `contador` y obteniendo la función `setContador`.

2. Al pulsar el botón "Incrementar", se dispara el manejador incrementar, el cual invoca a `setContador(contador + 1)`.
3. React detecta la alteración del estado, programa un nuevo renderizado del componente y actualiza únicamente el nodo de texto que contiene `{contador}` en el DOM virtual.

2. Eventos en React

¿Qué son los Eventos en React?

Los **Eventos** en React son mecanismos mediante los cuales el sistema detecta y reacciona a las acciones realizadas por el usuario en la interfaz gráfica (como clics, movimientos del ratón, pulsación de teclas o envíos de formularios). React implementa un sistema unificado denominado **SyntheticEvent** (Evento Sintético), el cual envuelve el evento nativo del navegador para garantizar un comportamiento idéntico y homogéneo en todos los exploradores web.

Características principales

- **Nomenclatura en CamelCase:** A diferencia de HTML estándar (`onclick`, `onsubmit`), en React los manejadores se escriben en *camelCase* (`onClick`, `onSubmit`, `onChange`).
- **Paso de funciones como manejadores:** En lugar de pasar una cadena de texto ejecutable, se pasa una referencia directa a una función JavaScript: `onClick={manejarClic}`.
- **Prevención explícita del comportamiento por defecto:** No se puede retornar `false` para detener el comportamiento estándar del navegador; se debe invocar explícitamente al método `e.preventDefault()`.
- **Delegación de eventos centralizada:** Por rendimiento, React no adjunta listeners de eventos a cada nodo del DOM real de forma individual, sino que delega y captura los eventos en el nodo raíz de la aplicación.

¿Para qué sirven?

Sirven para capturar las intenciones del usuario en el navegador y ejecutar la lógica de negocio correspondiente, como validar un formulario, alternar vistas, disparar peticiones HTTP o actualizar el estado.

```
import React, { useState } from 'react';

function FormularioRegistro() {
  const [nombre, setNombre] = useState('');
  const [mensaje, setMensaje] = useState('');

  const manejarCambio = (evento) => {
    setNombre(evento.target.value);
  };

  const manejarEnvio = (evento) => {
```

```

    evento.preventDefault(); // Detiene la recarga por defecto de la
página
    if (nombre.trim() === '') {
        setMensaje('Por favor, ingrese un nombre válido.');
```

```

    } else {
        setMensaje(`¡Bienvenido/a, ${nombre}!`);
        setNombre('');
    }
};

return (
    <div className="formulario-box">
        <form onSubmit={manejarEnvio}>
            <label htmlFor="inputNombre">Ingrese su nombre:</label>
            <input
                id="inputNombre"
                type="text"
                value={nombre}
                onChange={manejarCambio}
                placeholder="Ej: Juan Pérez"
            />
            <button type="submit">Enviar</button>
        </form>
        {mensaje && <p className="mensaje-alerta">{mensaje}</p>}
    </div>
);
}

export default FormularioRegistro;
```

Explicación del ejemplo:

1. `onChange={manejarCambio}`: Captura el evento de entrada en tiempo real; a través de `evento.target.value`, extrae lo que el usuario tipea y actualiza el estado `nombre`.
2. `onSubmit={manejarEnvio}`: Intercepta el evento de envío del formulario. Mediante `evento.preventDefault()`, evita que el navegador recargue la página completa.
3. Se actualiza el estado `mensaje`, renderizando condicionalmente la respuesta al usuario según los datos ingresados.

3. Hooks en React

¿Qué son los Hooks?

Introducidos en la versión 16.8 de React, los **Hooks** son funciones especiales que permiten "engancharse" (*hook into*) las capacidades de gestión de estado y el ciclo de vida de React directamente dentro de **componentes funcionales**, eliminando la necesidad de programar con componentes de clase (*class components*).

Características principales

- **Uso exclusivo en componentes funcionales:** No pueden ser utilizados dentro de componentes basados en clases ni en funciones regulares de JavaScript (salvo dentro de *Custom Hooks*).
- **Regla del nivel superior (Top Level):** Solo deben invocarse en el nivel superior del componente. No se deben colocar dentro de condicionales (if), bucles (for) o funciones anidadas para preservar el orden de las llamadas entre renderizados.
- **Prefijo obligatorio use:** Todos los hooks nativos y personalizados comienzan por convención con la palabra use (ej: `useState`, `useEffect`, `useContext`).
- **Composibilidad:** Permiten empaquetar y extraer lógica compleja con estado en funciones reutilizables (*Custom Hooks*).

¿Para qué sirven?

Permiten simplificar el código, evitar la duplicación de lógica de ciclo de vida dispersa en múltiples métodos (`componentDidMount`, `componentDidUpdate`), y fomentar la modularidad y el desacoplamiento en aplicaciones modernas.

¿Cuáles son los principales Hooks?

Hook	Propósito principal
useState	Permite declarar y gestionar variables de estado local dentro de un componente funcional.
useEffect	Ejecuta efectos secundarios (peticiones a APIs, suscripciones a eventos, manipulación directa del DOM o temporizadores) tras el renderizado. Reemplaza los ciclos de vida <code>componentDidMount</code> , <code>componentDidUpdate</code> y <code>componentWillUnmount</code> .
useContext	Permite suscribirse a un contexto de React para acceder a datos globales (ej: usuario autenticado, tema visual) sin tener que pasar <i>props</i> manualmente de padre a hijo a través de niveles intermedios (<i>prop drilling</i>).

useRef	Crea una referencia mutable que persiste durante todo el ciclo de vida del componente sin desencadenar un nuevo renderizado cuando su contenido cambia; comúnmente usado para acceder directamente a elementos del DOM.
useMemo	Memoriza el resultado del cálculo de una función costosa, recalculándolo únicamente si cambian sus dependencias.
useCallback	Memoriza la definición de una función entre renderizados para evitar regenerarla innecesariamente cuando se pasa a componentes hijos optimizados.
useReducer	Alternativa a useState orientada a gestionar estados locales con lógicas más complejas mediante acciones y funciones reductoras (similar al patrón Redux).

```
import React, { useState, useEffect } from 'react';

function CronometroSimple() {
  const [segundos, setSegundos] = useState(0);
  const [activo, setActivo] = useState(false);

  useEffect(() => {
    let intervalo = null;

    if (activo) {
      intervalo = setInterval(() => {
        setSegundos((segundosPrevios) => segundosPrevios + 1);
      }, 1000);
    } else {
      clearInterval(intervalo);
    }

    return () => clearInterval(intervalo);
  }, [activo]);

  return (
    <div className="cronometro-contenedor">
      <h2>Temporizador</h2>
    </div>
  );
}
```

```

    <p>Tiempo transcurrido: <strong>{segundos} s</strong></p>
    <button onClick={() => setActivo(!activo)}>
      {activo ? 'Pausar' : 'Iniciar'}
    </button>
    <button onClick={() => { setActivo(false); setSegundos(0); }}>
      Restablecer
    </button>
  </div>
);
}

export default CronometroSimple;

```

Explicación del ejemplo:

1. Se combinan dos hooks fundamentales: **useState** para el estado de los datos (segundos y activo) y **useEffect** para la tarea asíncrona del temporizador.
2. **useEffect** inicia un temporizador `setInterval` únicamente cuando `activo` es verdadero.
3. El arreglo de dependencias `[activo]` asegura que el efecto solo se configure nuevamente cuando el usuario presione el botón de Iniciar/Pausar.
4. La función de retorno dentro de **useEffect** (`return () => clearInterval(intervalo)`) garantiza la limpieza del intervalo de memoria al desmontar el componente o al cambiar el estado, previniendo fugas de memoria (*memory leaks*).